

GAINS REQUIREMENTS DOCUMENT

Team members: Jonathan Gallo, Murat Yildiz, Yaroslav, Aiden, Braden, Kevin

CSC 230 Project – Requirements WIP

Overview

- Stack: Next.js with React (frontend), Node.js with Express (backend), MongoDB (database). (Yes)
- R and RStudio used for writing and testing R code. (Yes)

Purpose

- Build a web app for running R analyses through a simple interface for social science students. (Yes)
- Allow social science students to easily generate statistical regression outputs using their own inputted data. (Yes)
- Allow social science students to view how the R code works and the context. (Yes)

Functional Requirements

- **Must Have:** Users receive R code to paste and can execute it. (Yes)
- **Must Have:** Users can upload datasets and associate them with their projects. (Yes)
- **Should Have:** Users can login and view their account/job history. (Yes)
- **Must Have:** System stores job history, status, and outputs for review and stores in database. (Needs to be finished)
- **Must Have:** Results (tables, text, plots) can be viewed or downloaded. (Needs to be finished)

Non-Functional Requirements

- Reliable and secure with clear error handling. (Yes)
- Easy to use, with a simple and responsive interface. (Yes)
- Reasonable performance with jobs completed in a short time. (Yes)
- Accessible through common web browsers (Chrome, Safari, etc.). (Yes)
- Accessible to students with disabilities (ADA compliance). (Needs to be finished)
- System should be able to support multiple users concurrently. (Yes)

User Stories

- As a student, I want to input data and receive linear regressions and the corresponding R code so that I can understand how the analysis works. (Yes)
- As a student, I want to download my results so that I can include them in my assignments or research. (Yes)
- As an instructor, I want to review outputs and logs to verify correctness so I can fairly evaluate students' work. (Yes)

- As a team member, I want a clear interface to collaborate effectively.

System Architecture

- Next.js with React (CSS Styling) provides the user interface for submitting and viewing jobs. (Yes)
- Express with Node.js manages requests, runs R code, and tracks jobs. (Yes)
- MongoDB stores job data, parameters, and references to outputs. (Needs more work)

Execution Strategy

- R code creation is created in the frontend using RCodeServices.js (/client). (Yes)
- Outputs such as tables and plots are created in the frontend. (Yes)

Milestones

- Scaffold frontend and backend setup. (Done)
- Create all required documents (Collab, Requirements, High Level Design, and Detailed Design) [Benchmark 1]. (Done)
- Get temporary working frontend. (Done)
- Create temporary JavaScript backend code (Node.js/Express placeholder for R execution) (Done)
- Finalize polish and prepare for demo. [Benchmark 2]. (Done)
- Add database via MongoDB for data storage. (Done)
- Login/Account system. (Done)
- Overall refactoring and optimization. (Needs more work)
- Full integration (database, login, polished UI), final testing, deployment, and presentation prep. [Benchmark 3]. (Done)

Definition of Done

- Users can submit R code with data and view accurate results. (Done)
- Outputs are saved, downloadable, and persist in MongoDB via account systems. (Needs more work)
- Interface is clear, reliable, and meets all functional requirements. (Done)
- All major user stories are satisfied and verified through testing. (Done)